Draw It or Lose It

Software Architecture Case Study | Kyle DeJong

An undergraduate software design study using the supplied Gaming Room scenario and domain model. The report proposes a browser-based multiplayer architecture and explains the requirements, platform choices, and shared-state decisions. The deliverable is design analysis.

Executive Summary

The Gaming Room is presented as seeking the services of Creative Technology Solutions (CTS) in developing a web-based version of their Android gaming app, Draw It or Lose It. The goal is to make the game available on multiple platforms, and for CTS to prepare a streamlined development process. This will include supporting multiple concurrent users across distributed platforms without compromising game behavior. The game will also incorporate team-based play, requiring unique game and team names and a centralized service to manage game instances. CTS will address these needs with object-oriented design principles and established software design patterns to support scalability, maintainability, and controlled access to game functionality.

Requirements

The Gaming Room has requested their Android game, Draw It or Lose It, be made into a web-based form for greater platform distribution. The existing game code and assets must therefore be translated into a chosen web-form. Further, the game must possess the ability to support multiple teams, each composed of multiple players, playing within the same game instances. Unique game and team names must therefore be implemented and tracked to avoid conflicts. The course model also calls for a single GameService instance within the application. This singleton requirement applies to an application instance; it does not establish a single authority across separate server processes.

Design Constraints

Draw It or Lose It will be web-based, thus operating on distributed environments that must remain synchronized. Multiple clients will connect to a centralized server that creates and moderates game instances. Because clients will communicate over a network rather than shared memory on a singular device, network latency will be a factor in coordinating synced game clients across teams, as well as delivering satisfactory performance to users on slower connections by minimizing unnecessary data exchange. As the game will operate with platform independence, no specific platform APIs will be used, but rather browser-based execution. As the game grows in use, scalability must support increased numbers of users and game sessions. Thus, design patterns and architectural decisions must avoid unnecessary technical overhead on the central server resources. As well, game clients must only have access to the data needed to play the game properly, with no ability to affect other clients' game states that is not moderated by central processing.

Proposed Architecture

Browser clients provide display and input. An authoritative application service validates requests, applies game rules, and returns state updates. Shared server-side persistence holds session, team, and player records. Media assets may be cached by clients to reduce repeated transfers. These are proposed responsibilities, not implemented or benchmarked components.

For an initial deployment, one application process can own active game state. Scaling to multiple processes requires explicit coordination: shared storage for persistent records, atomic updates and uniqueness constraints, and an agreed owner or coordination mechanism for each active session. Static counters and an in-process singleton are insufficient across processes. Transport details, concurrency behavior, and recovery procedures remain implementation decisions.

Domain Model

The following explains the supplied domain model and the responsibilities of its classes.

The game uses ProgramDriver as its execution entry point. A SingletonTester object will be used to test internal compliance with the singleton design pattern for GameService objects. The main base object type will be called Entity, which possesses both a private no-argument constructor and a public parameterized constructor, the latter requiring an id and name to be initialized. It contains an id, name, getter methods, and a data output function. It will be inherited by Game, Team, and Player objects which take on its members and methods. This structure demonstrates inheritance and encapsulation by containing common data (id and name) in a base class while its child classes keep their internal collections private and accessible only through methods.

A Game object will contain a private list of Team objects, a public constructor, and the ability to create a new team and an override of outputting pertinent Game information via toString(). A Team object contains a private list of type Player, a public constructor, and the means of adding new players and a toString() override as well. Player then contains only a public constructor and its own toString().

In the chain that descends from Entity, Game holds a list of Team which holds a list of Player and can create new instances of their associated types, and each relationship represents a zero-to-many association, from Game to Team and Team to Player. Each derived class requires its own id and name to be initialized, as well as a toString() override.

GameService is zero to many with Game, but does not inherit from Entity. It makes use of the singleton design pattern, with construction restricted through a private constructor. It holds the list of all games (type Game), and tracks via static variables the next valid Game, Player, and Team id. As such, it can return a handle to its own, private instance of itself, create new games, and look up games by id or name. As well, it tracks the list of total games in its count, as well as producing the next valid Player or Team id. Thus GameService acts as a centralized manager for game creation and identifier management.

This model describes local object relationships. Thread safety, transaction boundaries, globally unique identifiers, and conflict handling would need separate implementation and verification. A singleton does not automatically make access to its collections or counters safe under concurrency.

Platform Evaluation

The comparison below considers hosting, browser access, and development tooling. The recommendation is specific to this proposed web application; it is not a general ranking of operating systems.

macOS

A Unix-based environment can support development and hosting. For production, hosting availability, hardware choices, licensing terms, and administrator experience should guide selection rather than an assumed intrinsic scalability limit. Browser support must be tested against the intended browser versions and devices. Java, JavaScript, HTML, and CSS development can use Eclipse, IntelliJ IDEA, or Visual Studio Code, with browser tools for debugging.

Linux

Linux is a practical server choice for this proposal, with common web-server and development tooling. Many distributions have no base OS license charge, although hosting, support, and administration still cost money. Client compatibility depends primarily on browser capabilities; distribution differences can require additional checks, but do not inherently cause exponential testing complexity. Eclipse, IntelliJ IDEA, Visual Studio Code, and command-line build tools support development.

Windows

Windows offers familiar administration and development tools and can host web applications. Licensing and operating costs depend on the deployment rather than an assumption that Windows always incurs greater overhead. Client testing should cover intended Windows browsers. Java and web development can use Eclipse, IntelliJ IDEA, Visual Studio Code, and browser debugging tools.

Mobile Devices

Mobile devices are client and test targets for this design, rather than production server hosts or the primary development environment. Screen size, touch input, responsive layout, memory, power, and changing network conditions affect usability. Testing must cover actual target devices and browser behavior; sharing a web codebase does not guarantee identical behavior.

Recommendations

Operating Platform

A Linux-based server is the recommended starting point for Draw It or Lose It. It fits the proposed web workload and offers flexible hosting and tooling choices. A browser client allows access from Windows, macOS, Linux, and mobile devices without a separate native implementation for each. Consistent behavior remains a testing goal, not a consequence guaranteed by the server OS.

Operating Systems Architecture

Use a client-server model. The application service manages game logic, validates actions, and maintains authoritative game state, while clients handle display and input. Linux supports multiple users and processes, but application-level coordination is still required when scaling game services. Separate processes do not share a singleton or static identifier counters.

Storage Management

Use structured server-side storage for persistent session, team, and player data. Define the intended scope of uniqueness, such as unique game names across the service and unique team names within a game, then enforce it with appropriate database constraints and atomic operations. Concurrent requests must not rely on an unprotected check-then-insert sequence. A logically authoritative store may be replicated; backups and recovery procedures still need to be designed and tested. Storage alone does not keep clients synchronized: the application must deliver and reconcile updates.

Memory Management

The operating system allocates memory to processes and provides process isolation. Within a shared process, the application must separate session data and control access; OS process isolation does not automatically isolate individual game sessions. Clients should retain only the state needed for interaction. Caching media can improve response time at the cost of client memory and storage, while on-demand delivery increases network dependence. Limits and cleanup policies for inactive sessions need implementation.

Distributed Systems and Networks

Clients communicate with the authoritative service using web protocols. Network latency, loss, and disconnection must be treated as normal operating conditions. Preserve state for a defined session lifetime and support reconnecting clients with a current state snapshot. Requests need validation against current session state; retry handling and stale or duplicate commands require explicit rules. Reconnection and uninterrupted play are design goals whose limits depend on session timing and recovery behavior, not guarantees that a client failure cannot affect teammates.

Security

Protect credentials and stored data, authorize actions server-side, and encrypt network traffic. Clients should receive only the information needed for their session, and the service must verify access before accepting state changes. Restrict access to application files and storage through server and operating-system permissions. Centralized validation reduces reliance on client trust, but does not remove the need for secure implementation, configuration, and testing.

Technical References

PostgreSQL documentation, Constraints: https://www.postgresql.org/docs/18/ddl-constraints.html — an example of database-enforced uniqueness, not a claim that PostgreSQL was implemented here.

Visual Studio Code FAQ: https://code.visualstudio.com/docs/supporting/faq — supported development platforms.